HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

THỰC TẬP CƠ SỞ

BÁO CÁO TUẦN

Đề tài:

Xây dựng website tuyển dụng việc làm

Giảng viên hướng dẫn	**: Kim Ngọc Bách**
Họ và tên sinh viên	**: Đỗ Bình Minh**
Ngày sinh	**: 24/12/2005**
Số điện thoại	**: 0329279863**
Mã sinh viên	**: B23DCCN542**
Lớp	**: D23CQCN10-B**
Nhóm lớp	**: 11**

Hà Nội – 2026

Mục lục

1. Tóm tắt các công việc đã thực hiện trong tuần 3
2. Chi tiết các công việc đã thực hiện trong tuần 3
2.1. Tích hợp bộ soạn thảo TinyMCE cho form mô tả chi tiết 3
2.1.1. Giới thiệu TinyMCE 3
2.1.2. Xử lý bên Frontend 4
2.1.3. Xử lý bên Backend 6
2.1.4. Kết quả 7
2.2. Tạo đường dẫn bảo mật bên Frontend 7
2.3. Trang thêm mới công việc (Nhà tuyển dụng) 8
2.3.1. Xử lý bên phía Frontend 8
2.3.2. Xử lý phía Backend 11
2.3.3. Kết quả 14
2.4. Trang quản lý công việc (Nhà tuyển dụng) 16
2.4.1. Xử lý bên phía Frontend 16
2.4.2 Xử lý bên phía Backend 16
2.4.3. Kết quả 19
3. Kết quả đạt được 19
4. Khó khăn gặp phải 20
5. Hướng xử lý 20
6. Kế hoạch tuần tiếp theo 21

1. Tóm tắt các công việc đã thực hiện trong tuần

Trong tuần này, công việc trọng tâm của em là tiếp tục hoàn thiện các chức năng chính cho website tuyển dụng việc làm, tập trung vào cả giao diện và xử lý nghiệp vụ giữa frontend với backend. Mục tiêu đặt ra là bổ sung các tính năng phục vụ nhà tuyển dụng trong quá trình đăng tuyển và quản lý công việc, đồng thời tăng tính hoàn thiện và bảo mật cho hệ thống. Cụ thể, em đã thực hiện tích hợp bộ soạn thảo TinyMCE cho phần mô tả chi tiết, xây dựng cơ chế upload ảnh, tạo middleware kiểm tra đăng nhập để bảo vệ một số trang, phát triển chức năng thêm mới công việc và hiển thị danh sách công việc của nhà tuyển dụng. Những phần này giúp hệ thống tiến thêm một bước trong việc hoàn thiện luồng đăng tuyển, quản lý dữ liệu tuyển dụng và tạo nền tảng để tiếp tục phát triển các chức năng chỉnh sửa, xóa, tìm kiếm và phân trang trong giai đoạn tiếp theo.

2. Chi tiết các công việc đã thực hiện trong tuần

2.1. Tích hợp bộ soạn thảo TinyMCE cho form mô tả chi tiết

2.1.1. Giới thiệu TinyMCE

TinyMCE là một **trình soạn thảo văn bản WYSIWYG** chạy trên web. WYSIWYG nghĩa là *"What You See Is What You Get"* — tức là khi gõ và định dạng nội dung, bạn sẽ thấy gần giống kết quả cuối cùng ngay trên giao diện.

Nói ngắn gọn, TinyMCE giúp biến một ô nhập văn bản thường thành **editor giống Word thu nhỏ** trong website, có thể:

- in đậm, in nghiêng, gạch chân
- căn lề, tạo danh sách
- chèn ảnh, bảng, link
- đổi màu chữ, cỡ chữ
- soạn nội dung HTML dễ hơn

TinyMCE thường được dùng trong:

- trang quản trị viết bài blog
- hệ thống CMS
- form nhập mô tả sản phẩm
- phần bình luận hoặc soạn nội dung email

2.1.2. Xử lý bên Frontend

- Lệnh cài đặt: npm install tinymce @tinymce/tinymce-react

- Code:

```
let description = "";
if(editorRef.current) {
  description = (editorRef.current as any).getContent();
}
```

Đoạn này dùng để **lấy nội dung đang có trong TinyMCE editor** và gán vào biến description.

- let description = "";
  tạo biến rỗng ban đầu.
- if (editorRef.current)
  kiểm tra editor đã được khởi tạo chưa.
- editorRef.current.getContent()
  lấy nội dung hiện tại trong editor.

=> nếu editor tồn tại thì lấy nội dung của nó đưa vào description

```
<EditorMCE
  editorRef={editorRef}
  value={infoCompany.description}
/>
```

Dòng này có tác dụng **truyền giá trị mô tả ban đầu vào editor**.

- editorRef={editorRef}
  gắn ref cho TinyMCE để sau này có thể gọi editor từ code, ví dụ getContent().
- value={infoCompany.description}
  đặt nội dung của editor bằng giá trị description trong infoCompany.

=> editorRef để điều khiển/lấy dữ liệu từ editor, còn value để hiển thị nội dung mô tả lên editor.

```
/* eslint-disable @typescript-eslint/no-explicit-any */
import { Editor } from "@tinymce/tinymce-react"

export const EditorMCE = (props: {
  editorRef: any,
  value: string
}) => {
  const { editorRef, value } = props;

  return (
    <>
      <Editor
        apiKey={process.env.NEXT_PUBLIC_API_TINYMCE}
        onInit={ (_evt, editor) => editorRef.current = editor }
        initialValue={value}
        init={{
          height: 500,
          plugins: [
            'advlist', 'autolink', 'lists', 'link', 'image', 'charmap', 'preview',
            'anchor', 'searchreplace', 'visualblocks', 'code', 'fullscreen',
            'insertdatetime', 'media', 'table', 'code', 'help', 'wordcount'
          ],
          toolbar: `undo redo | blocks | bold italic forecolor | alignleft aligncenter alignright
alignjustify | bullist numlist outdent indent | removeformat | help`,
          images_upload_url: `${process.env.NEXT_PUBLIC_API_URL}/upload/image`
        }}
      />
    </>
  )
}
```

Đây là một **component TinyMCE dùng lại được** để hiển thị ô soạn thảo văn bản.

Tác dụng từng phần:

- import { Editor } from "@tinymce/tinymce-react"
  nhập component TinyMCE cho React.
- editorRef: any
  nhận ref từ component cha để có thể lấy nội dung editor sau này.
- value: string
  nhận nội dung ban đầu đưa vào editor.
- const { editorRef, value } = props;
  tách dữ liệu từ props.
- apiKey={process.env.NEXT_PUBLIC_API_TINYMCE}
  lấy API key từ biến môi trường.
- onInit={(_evt, editor) => editorRef.current = editor}
  khi editor khởi tạo xong thì lưu editor vào editorRef.current.

- initialValue={value}
  đặt nội dung ban đầu cho editor.
- init={{ ... }}
  cấu hình editor:
  - height: 500 → chiều cao 500
  - plugins → bật các chức năng như ảnh, bảng, link, code, fullscreen...
  - toolbar → thanh công cụ có các nút định dạng
  - images_upload_url → nơi TinyMCE gửi ảnh lên server khi upload

=> Component này dùng để **hiển thị TinyMCE, nhận nội dung ban đầu, lưu editor vào ref, và cấu hình các chức năng như toolbar, plugin, upload ảnh**

2.1.3. Xử lý bên Backend

```
router.use("/upload", uploadRoutes)
```

Định nghĩa ra router: nói với Express rằng: mọi request bắt đầu bằng /upload sẽ chuyển cho uploadRoutes xử lý.

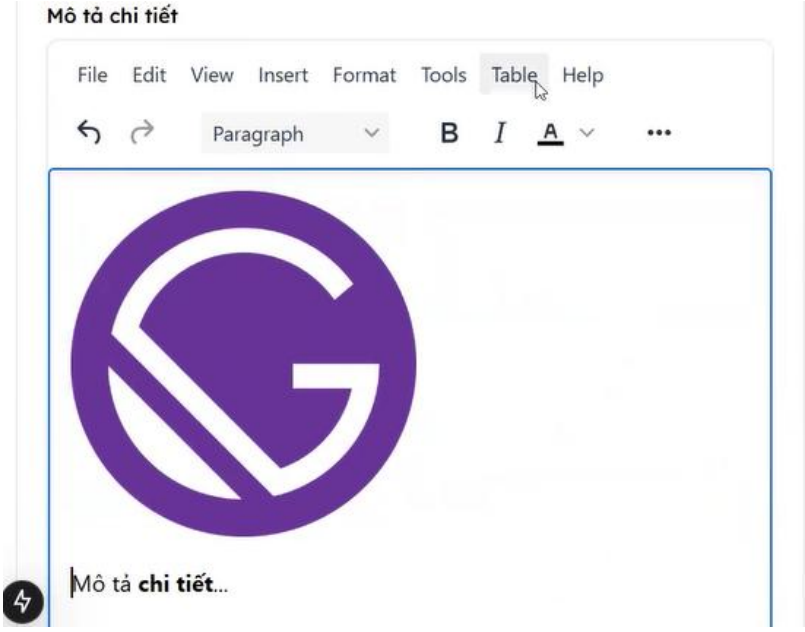
```
import { Router } from "express";
import * as uploadController from "../controllers/upload.controller";
import multer from "multer";
import { storage } from "../helpers/cloudinary.helper";

const router = Router();

const upload = multer({ storage: storage });

router.post(
  "/image",
  upload.single("file"),
  uploadController.imagePost
)

export default router;
```

tạo một API để nhận file ảnh upload lên server.

```
import { Request, Response } from "express";

export const imagePost = async (req: Request, res: Response) => {
  res.json({
    location: req?.file?.path
  });
}
```

Đây là controller imagePost, dùng để xử lý request upload ảnh và trả về đường dẫn ảnh vừa upload.

2.1.4. Kết quả



Form mô tả chi tiết đã có những chức năng bổ trợ như word, có thể nhập nội dung, tải ảnh lên…

2.2. Tạo đường dẫn bảo mật bên Frontend

```
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

export function middleware(request: NextRequest) {
  const token = request.cookies.get('token')?.value;
  if(token) {
    return NextResponse.next();
  } else {
    return NextResponse.redirect(new URL('/', request.url));
  }
}

export const config = {
  matcher: [
    '/user-manage/:path*',
    '/company-manage/:path*',
  ],
}
```

Đây là file middleware của Next.js, dùng để kiểm tra người dùng đã đăng nhập chưa trước khi cho vào một số trang.

Cụ thể:

- middleware là **middleware**
- nó đọc cookie token
- nếu có token thì cho đi tiếp
- nếu không có thì chuyển về trang chủ /

2.3. Trang thêm mới công việc (Nhà tuyển dụng)

2.3.1. Xử lý bên phía Frontend

```
// Register the plugin
registerPlugin(
  FilePondPluginFileValidateType,
  FilePondPluginImagePreview
);

export const FormCreate = () => {
  const editorRef = useRef(null);
  const [images, setImages] = useState<any[]>([]);

  useEffect(() => {
    const validator = new JustValidate("#createForm");

    validator
      .addField('#title', [
        {
          rule: 'required',
          errorMessage: 'Vui lòng nhập tên công việc!'
        },
      ])
      .addField('#salaryMin', [
        {
          rule: 'minNumber',
          value: 0,
          errorMessage: 'Vui lòng nhập mức lương >= 0'
        },
      ])
      .addField('#salaryMax', [
        {
          rule: 'minNumber',
          value: 0,
          errorMessage: 'Vui lòng nhập mức lương >= 0'
        },
      ])
  }, []);
```

Đây là phần code trong component FormCreate, dùng để khởi tạo form tạo job và kiểm tra dữ liệu nhập bằng JustValidate.

Cụ thể:

- registerPlugin(...) là đăng ký plugin cho FilePond
- editorRef dùng để tham chiếu TinyMCE
- images là state lưu danh sách ảnh
- useEffect(...) dùng để khởi tạo validator khi form render
- các lệnh .addField(...) là đặt rule kiểm tra cho các ô nhập như title, salaryMin, salaryMax.

```
const handleSubmit = (event: any) => {
  event.preventDefault();
  const title = event.target.title.value;
  const salaryMin = event.target.salaryMin.value;
  const salaryMax = event.target.salaryMax.value;
  const position = event.target.position.value;
  const workingForm = event.target.workingForm.value;
  const technologies = event.target.technologies.value;
  let description = "";
  if (editorRef.current) {
    description = (editorRef.current as any).getContent();
  }

  // Tạo FormData
  const formData = new FormData();
  formData.append("title", title);
  formData.append("salaryMin", salaryMin);
  formData.append("salaryMax", salaryMax);
  formData.append("position", position);
  formData.append("workingForm", workingForm);
  formData.append("technologies", technologies);
  formData.append("description", description);

  // Images
  if(images.length > 0) {
    for (const image of images) {
      formData.append("images", image.file);
    }
  }
  // End Images

  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/create`, {
    method: "POST",
    body: formData,
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "error") {
        toast.error(data.message);
      }

      if(data.code == "success") {
        toast.success(data.message);
        event.target.reset();
        setImages([]);
      }
    })
}
```

Có thể nói ngắn gọn như này:

Đây là hàm handleSubmit, dùng để lấy dữ liệu từ form, đóng gói thành FormData, gửi lên API tạo job, rồi xử lý kết quả trả về.

Cụ thể:

- lấy các giá trị người dùng nhập
- lấy nội dung từ TinyMCE
- thêm dữ liệu và ảnh vào FormData
- gửi POST lên API /company/job/create
- nếu thành công thì hiện thông báo và reset form
- nếu lỗi thì hiện thông báo lỗi.

```
        id={id}
        init={{
          height: 500,
          plugins: [
            'advlist', 'autolink', 'lists', 'link', 'image', 'charmap', 'preview',
            'anchor', 'searchreplace', 'visualblocks', 'code', 'fullscreen',
            'insertdatetime', 'media', 'table', 'code', 'help', 'wordcount'
          ],
          toolbar: `undo redo | blocks | bold italic forecolor | alignleft aligncenter alignright
alignjustify | bullist numlist outdent indent | removeformat | help`,
          images_upload_url: `${process.env.NEXT_PUBLIC_API_URL}/upload/image`
        }}
      />
    </>
  )
}
```

Đây là phần cấu hình init của TinyMCE, dùng để thiết lập giao diện và chức năng của editor.

```
export const positionList = [
  {
    label: "Intern",
    value: "intern",
  },
  {
    label: "Fresher",
    value: "fresher",
  },
  {
    label: "Junior",
    value: "junior",
  },
  {
    label: "Middle",
    value: "middle",
  },
  {
    label: "Senior",
    value: "senior",
  },
  {
    label: "Manager",
    value: "manager",
  }
];

export const workingFormList = [
  {
    label: "Tại văn phòng",
    value: "office",
  },
  {
    label: "Làm từ xa",
    value: "remote",
  },
  {
    label: "Linh hoạt",
    value: "flexible",
  },
];
```

Đây là file hằng số dữ liệu, dùng để khai báo sẵn danh sách lựa chọn cho form.

2.3.2. Xử lý phía Backend

```
router.post(
  "/job/create",
  authMiddleware.verifyTokenCompany,
  upload.array("images", 8),
  companyController.createJobPost
)
```

Đây là route POST /job/create, dùng để tạo mới một job.

```
import mongoose from "mongoose";

const schema = new mongoose.Schema(
  {
    companyId: String,
    title: String,
    salaryMin: Number,
    salaryMax: Number,
    position: String,
    workingForm: String,
    technologies: Array,
    description: String,
    images: Array
  },
  {
    timestamps: true, // Tự động sinh ra trường createdAt và updatedAt
  }
);

const Job = mongoose.model('Job', schema, "jobs");

export default Job;
```

Đây là file model Mongoose, dùng để định nghĩa cấu trúc dữ liệu của Job trong MongoDB.

Cụ thể:

- schema là **schema** mô tả các trường của job như title, salaryMin, position, description, images...
- timestamps: true là tự thêm createdAt và updatedAt
- mongoose.model("Job", schema, "jobs") là tạo **model Job** để thao tác với collection jobs.

```
 import bcrypt from "bcryptjs";
 import jwt from "jsonwebtoken";
 import { AccountRequest } from "../interfaces/request.interface";
+import Job from "../models/job.model";

 export const registerPost = async (req: Request, res: Response) => {
   const existAccount = await AccountCompany.findOne({
@@ -95,3 +96,35 @@ export const profilePatch = async (req: AccountRequest, res: Response) => {
     message: "Cập nhật thành công!"
   })
 }
+
+export const createJobPost = async (req: AccountRequest, res: Response) => {
+  try {
+    req.body.companyId = req.account.id;
+    req.body.salaryMin = req.body.salaryMin ? parseInt(req.body.salaryMin) : 0;
+    req.body.salaryMax = req.body.salaryMax ? parseInt(req.body.salaryMax) : 0;
+    req.body.technologies = req.body.technologies ? req.body.technologies.split(", ") : [];
+    req.body.images = [];
+
+    // Xử lý mảng images
+    if(req.files) {
+      for (const file of req.files as any[]) {
+        req.body.images.push(file.path);
+      }
+    }
+    // Hết Xử lý mảng images
+
+    const newRecord = new Job(req.body);
+    await newRecord.save();
+
+    res.json({
+      code: "success",
+      message: "Tạo công việc thành công!"
+    })
+  } catch (error) {
+    console.log(error);
+    res.json({
+      code: "error",
+      message: "Dữ liệu không hợp lệ!"
+    })
+  }
+}
```

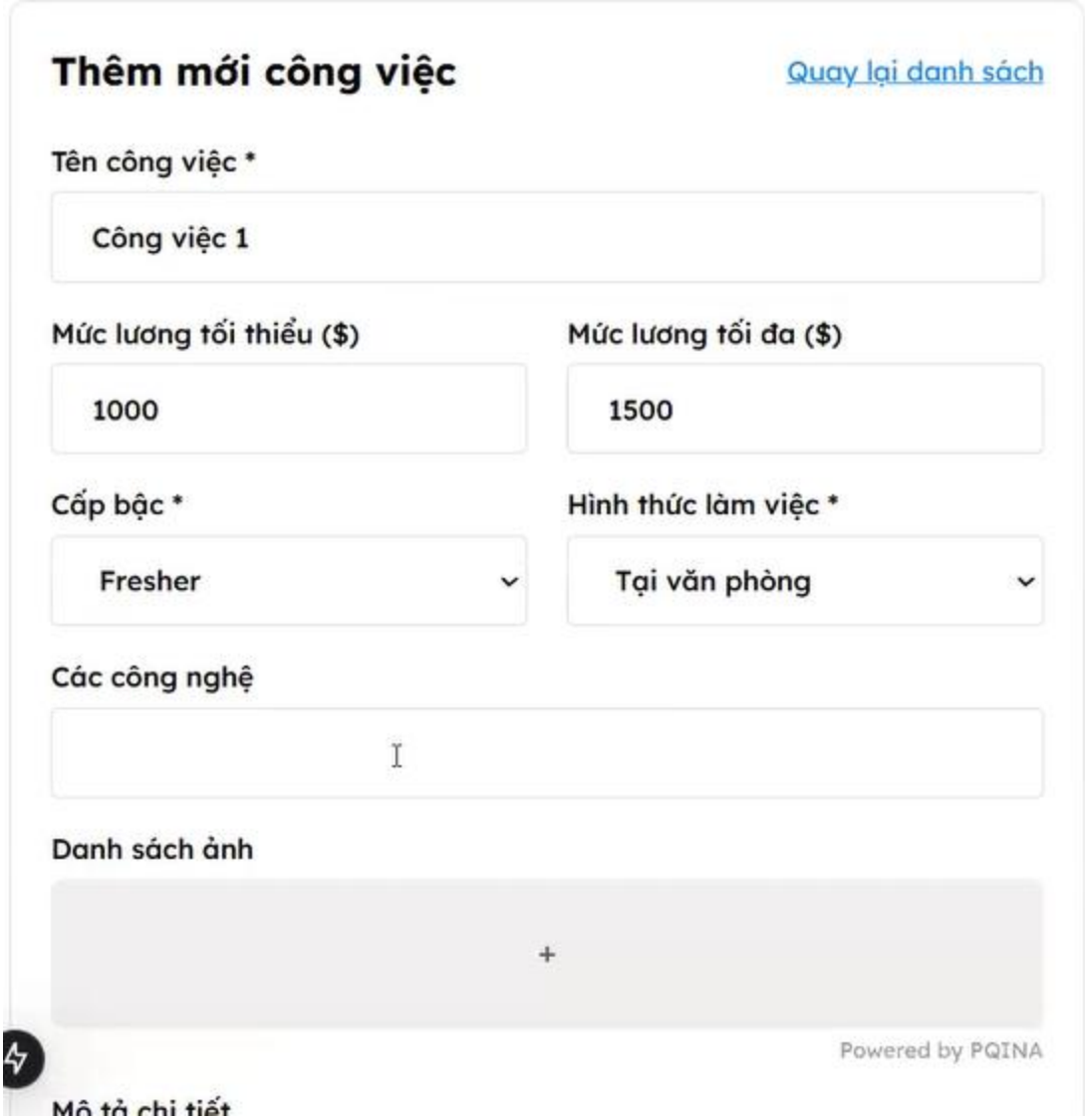
Đây là controller createJobPost, dùng để xử lý tạo mới một bài đăng tuyển dụng và lưu vào MongoDB.

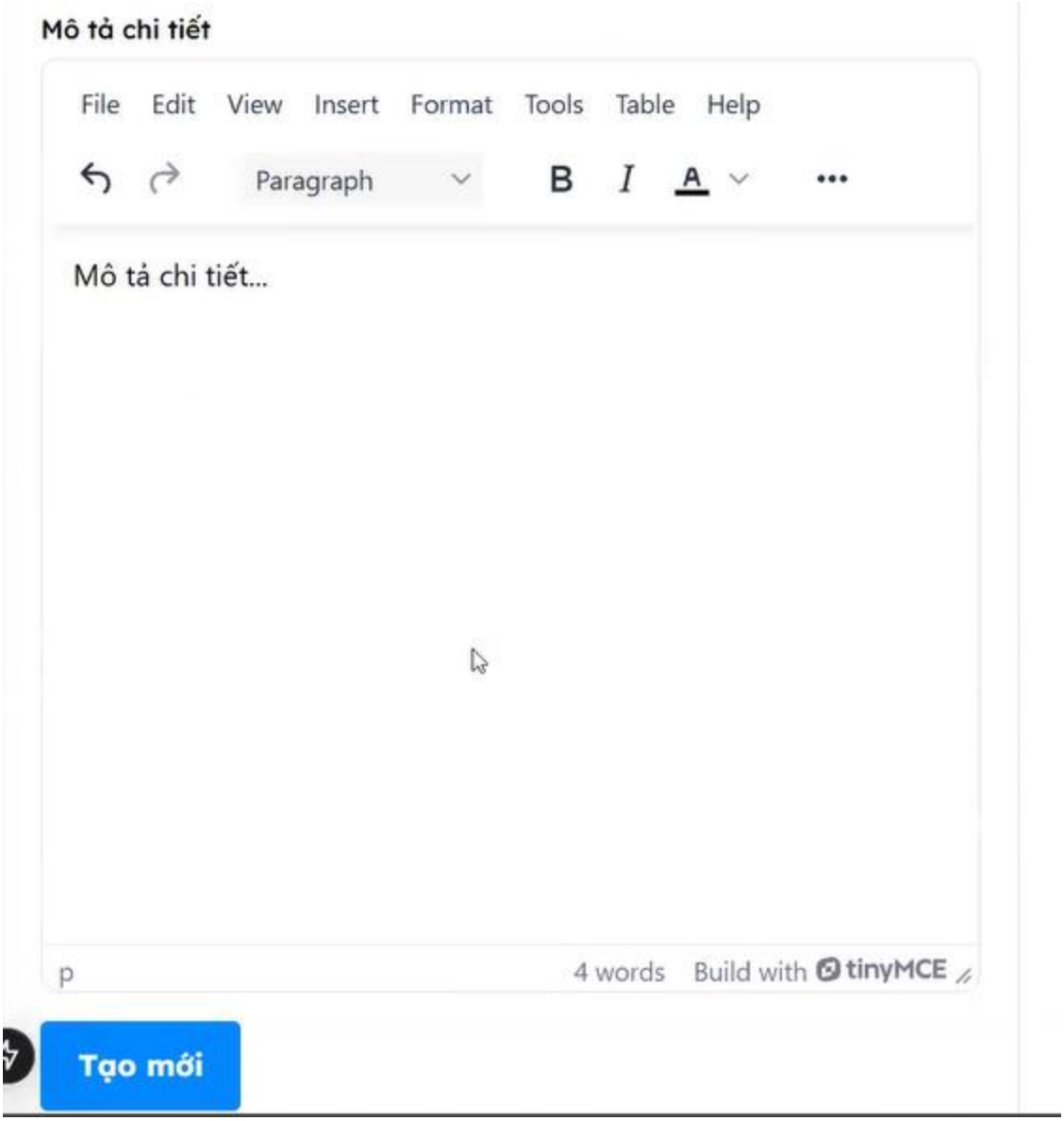
Cụ thể:

- lấy companyId từ tài khoản đang đăng nhập
- chuyển salaryMin, salaryMax sang số
- tách technologies thành mảng

- lấy các ảnh upload và đưa vào images
- tạo new Job(req.body) rồi save()
- trả kết quả thành công hoặc lỗi.

2.3.3. Kết quả






```
_id: ObjectId('68fb79757e5768f8f6e5f1e8')
companyId : "68f23eb24e45470e1672f80d"
title : "Công việc 1"
salaryMin : 1000
salaryMax : 1500
position : "fresher"
workingForm : "office"
▸ technologies : Array (3)
description : "<p>M&ocirc; tả chi tiết...</p>"
▸ images : Array (2)
createdAt : 2025-10-24T13:04:53.376+00:00
updatedAt : 2025-10-24T13:04:53.376+00:00
__v : 0
```

Sau khi nhập thông tin công việc và ấn thêm mới, dữ liệu đã được lưu trên CSDL online

2.4. Trang quản lý công việc (Nhà tuyển dụng)

2.4.1. Xử lý bên phía Frontend

```
useEffect(() => {
  fetch(`${process.env.NEXT_PUBLIC_API_URL}/company/job/list`, {
    method: "GET",
    credentials: "include", // Gửi kèm cookie
  })
    .then(res => res.json())
    .then(data => {
      if(data.code == "success") {
        setJobList(data.jobs);
      }
    })
}, []);
```

Đây là useEffect dùng để gọi API lấy danh sách job của công ty khi component được mở lần đầu.

Cụ thể:

- gửi request GET đến /company/job/list
- credentials: "include" để gửi kèm cookie
- nếu API trả về thành công thì lưu danh sách job vào setJobList(data.jobs).

2.4.2 Xử lý bên phía Backend

```
router.get(
  "/job/list",
  authMiddleware.verifyTokenCompany,
  companyController.listJob
)
```

Đây là route GET /job/list, dùng để lấy danh sách job của công ty.

Cụ thể:

- authMiddleware.verifyTokenCompany là **middleware kiểm tra đăng nhập/xác thực công ty**
- companyController.listJob là **controller xử lý lấy danh sách job**.

```
import AccountUser from "../models/account-user.model";
import { AccountRequest } from "../interfaces/request.interface";
import AccountCompany from "../models/account-company.model";
import City from "../models/city.model";

export const verifyTokenUser = async (
  req: AccountRequest,
@@ -84,6 +85,15 @@ export const verifyTokenCompany = async (

    req.account = existAccount;

    if(existAccount.city) {
      const city = await City.findOne({
        _id: existAccount.city
      })
      if(city) {
        req.account.companyCity = city.name;
      }
    }

    next();
  } catch (error) {
    res.clearCookie("token");
```

Đây là file middleware xác thực công ty, dùng để kiểm tra token đăng nhập và bổ sung thêm thông tin công ty vào req.account trước khi chuyển sang bước xử lý tiếp theo.

Cụ thể :

- dùng City là **model**
- tìm thành phố theo existAccount.city
- nếu tìm thấy thì gán req.account.companyCity = city.name

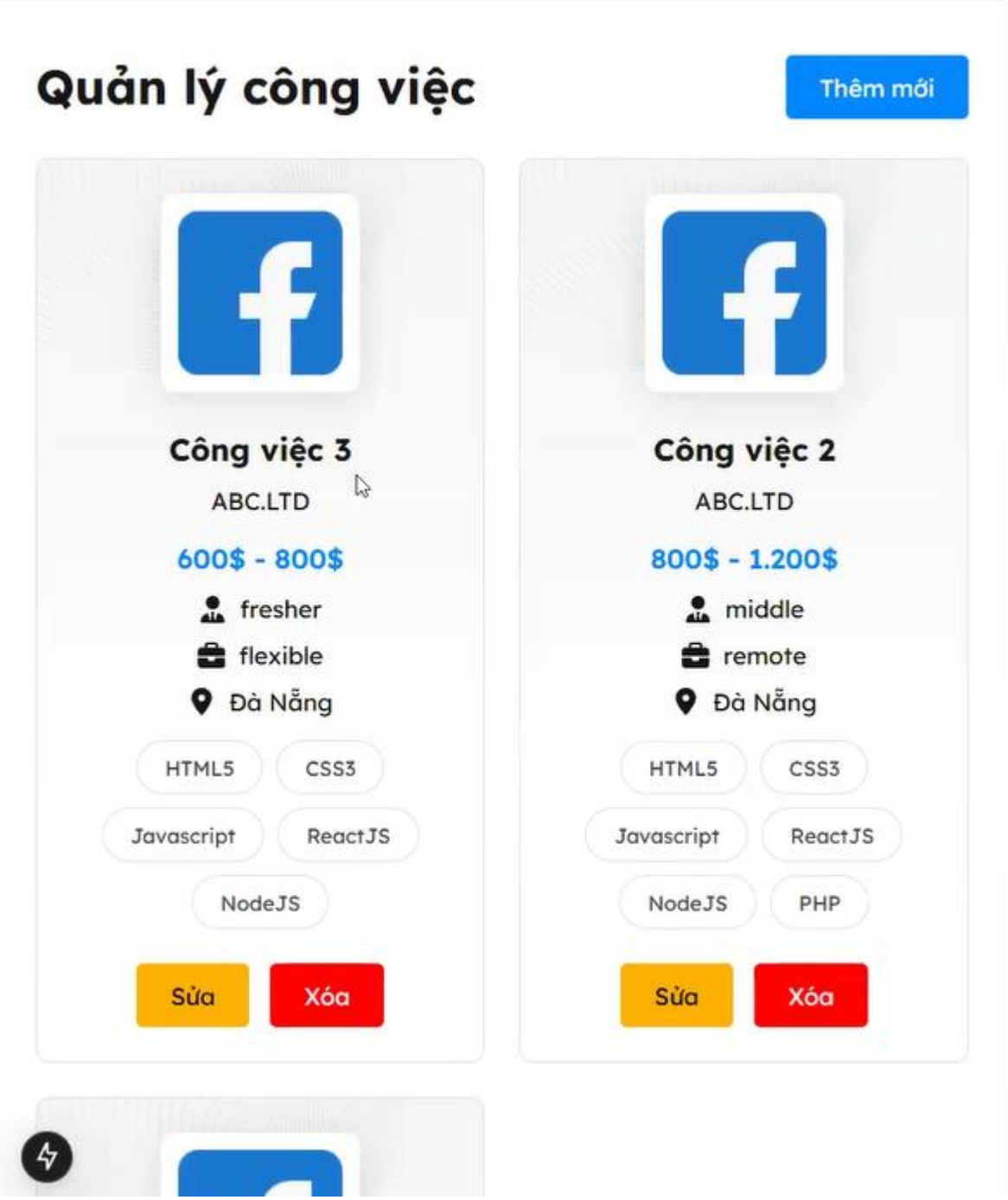
```
export const listJob = async (req: AccountRequest, res: Response) => {
  try {
    const companyId = req.account.id;

    const jobs = await Job
      .find({
        companyId: companyId
      })
      .sort({
        createdAt: "desc"
      })

    const dataFinal = [];

    for (const item of jobs) {
      dataFinal.push({
        id: item.id,
        companyLogo: req.account.logo,
        title: item.title,
        companyName: req.account.companyName,
        salaryMin: item.salaryMin,
        salaryMax: item.salaryMax,
        position: item.position,
        workingForm: item.workingForm,
        companyCity: req.account.companyCity,
        technologies: item.technologies,
      });
    }

    res.json({
      code: "success",
      message: "Thành công!",
      jobs: dataFinal
    })
  } catch (error) {
    console.log(error);
    res.json({
      code: "error",
      message: "Dữ liệu không hợp lệ!"
    })
  }
}
```

Đây là controller listJob, dùng để lấy danh sách job của công ty đang đăng nhập rồi trả về cho frontend.

Cụ thể:

- lấy companyId từ req.account
- tìm các job của công ty trong model Job
- sắp xếp theo createdAt giảm dần
- tạo lại dữ liệu cần trả về
- cuối cùng res.json(...) trả danh sách job về client.

2.4.3. Kết quả



Danh sách công việc đã được hiển thị để nhà tuyển dụng quản lý (Thêm, sửa, xóa…)

3. Kết quả đạt được

Trong tuần này, em đã hoàn thành một số chức năng quan trọng của website tuyển dụng việc làm. Trước hết, em đã tích hợp thành công bộ soạn thảo TinyMCE vào form mô tả chi tiết, giúp ô nhập liệu trở nên trực quan hơn, hỗ trợ định dạng văn bản, chèn liên kết,

bảng và tải ảnh lên hệ thống. Đồng thời, phía backend cũng đã xây dựng API upload ảnh để phục vụ cho editor hoạt động ổn định.

Bên cạnh đó, em đã triển khai middleware ở phía frontend để kiểm tra trạng thái đăng nhập của người dùng thông qua cookie token. Nhờ đó, một số trang cần bảo vệ chỉ cho phép truy cập khi người dùng đã đăng nhập, góp phần tăng tính an toàn cho hệ thống.

Ngoài ra, em đã xây dựng được chức năng thêm mới công việc dành cho nhà tuyển dụng. Form thêm mới đã hỗ trợ kiểm tra dữ liệu đầu vào, nhập mô tả bằng TinyMCE, tải ảnh lên, đóng gói dữ liệu dưới dạng FormData và gửi về backend. Phía server cũng đã xử lý lưu dữ liệu bài đăng tuyển dụng vào MongoDB thành công. Sau khi thêm mới, dữ liệu đã được lưu trên cơ sở dữ liệu online.

Cuối cùng, em đã thực hiện chức năng quản lý công việc của nhà tuyển dụng, bao gồm gọi API lấy danh sách job từ backend và hiển thị ra giao diện để phục vụ quản lý. Danh sách công việc đã được hiển thị thành công, tạo nền tảng cho các thao tác tiếp theo như sửa hoặc xóa bài đăng.

4. Khó khăn gặp phải

Trong quá trình thực hiện, em gặp một số khó khăn cả ở frontend lẫn backend. Với TinyMCE, khó khăn lớn nhất là việc tích hợp editor vào form React/Next.js sao cho vừa hiển thị đúng dữ liệu ban đầu, vừa lấy được nội dung hiện tại để gửi đi khi submit. Ngoài ra, việc cấu hình upload ảnh từ TinyMCE về server cũng cần xử lý đồng bộ giữa frontend và backend.

Ở phần tạo job, em gặp khó khăn trong việc kết hợp nhiều thành phần trong cùng một form như TinyMCE, FilePond, JustValidate và FormData. Việc kiểm tra dữ liệu đầu vào, chuyển đổi kiểu dữ liệu như salaryMin, salaryMax, cũng như xử lý danh sách ảnh và công nghệ sử dụng đòi hỏi phải cẩn thận để tránh lỗi khi lưu dữ liệu.

Đối với chức năng bảo mật và quản lý danh sách công việc, khó khăn nằm ở việc kiểm tra token, truyền cookie đúng cách và đảm bảo frontend có thể gọi API kèm credentials: "include" để backend xác thực chính xác. Bên cạnh đó, em cũng phải kiểm tra lại luồng xử lý giữa middleware, route và controller để dữ liệu trả về đúng như mong muốn.

5. Hướng xử lý

Để khắc phục các khó khăn trên, em đã chia nhỏ từng chức năng để xử lý. Với TinyMCE, em tách phần editor thành component riêng, sử dụng editorRef để lấy nội dung từ editor và truyền giá trị ban đầu qua props, từ đó giúp việc tái sử dụng và quản lý editor rõ ràng hơn. Đồng thời, em cấu hình riêng API upload ảnh ở backend để phục vụ việc chèn ảnh trong nội dung mô tả.

Với form tạo công việc, em sử dụng JustValidate để kiểm tra dữ liệu đầu vào ngay trên giao diện, sau đó dùng FormData để đóng gói cả dữ liệu văn bản lẫn ảnh trước khi gửi lên server. Ở backend, em chuẩn hóa dữ liệu trước khi lưu, như ép kiểu lương về dạng số, tách danh sách công nghệ thành mảng và xử lý danh sách ảnh upload một cách thống nhất. Cách này giúp giảm lỗi khi lưu vào MongoDB.

Đối với chức năng bảo mật và quản lý công việc, em sử dụng middleware để kiểm tra token trước khi cho phép truy cập vào các route cần xác thực. Đồng thời, em kiểm tra lại luồng request/response giữa frontend và backend, bổ sung việc gửi cookie đúng cách và kiểm tra dữ liệu trả về từ API để danh sách công việc hiển thị chính xác hơn.

6. Kế hoạch tuần tiếp theo

Trong tuần tiếp theo, em dự kiến tiếp tục phát triển và hoàn thiện một số chức năng mới của website tuyển dụng việc làm. Cụ thể, em sẽ triển khai tính năng **phân trang** ở cả phía **frontend** và **backend** để việc hiển thị danh sách dữ liệu như công việc tuyển dụng trở nên rõ ràng, dễ theo dõi và tối ưu hơn khi số lượng bản ghi tăng lên.

Bên cạnh đó, em sẽ xây dựng chức năng **sửa công việc** dành cho nhà tuyển dụng, bao gồm giao diện chỉnh sửa ở frontend và xử lý cập nhật dữ liệu tương ứng ở backend. Sau đó, em tiếp tục thực hiện chức năng **xóa công việc** để nhà tuyển dụng có thể quản lý bài đăng linh hoạt hơn.

Ngoài các chức năng quản lý công việc, em cũng dự kiến phát triển thêm một số thành phần giao diện ở trang người dùng như **khối menu trên header**, **khối tìm kiếm ở trang chủ** và **khối nhà tuyển dụng hàng đầu**. Các phần này sẽ góp phần hoàn thiện giao diện tổng thể, hỗ trợ người dùng tra cứu thông tin thuận tiện hơn và nâng cao trải nghiệm sử dụng website.

Trong quá trình thực hiện, em cũng sẽ kết hợp kiểm tra lại các chức năng đã làm ở tuần trước để đảm bảo hệ thống hoạt động ổn định, đồng bộ giữa frontend và backend, đồng thời hạn chế các lỗi phát sinh khi tích hợp chức năng mới.